

Aula 4: Comandos Condicionais e Manipulação de Strings com split()

Luís Felipe

UFF

01 de Setembro de 2025

Comandos condicionais

- Um **comando condicional** é aquele que permite decidir se um determinado bloco de comandos deve ou não ser executado, a partir do resultado de uma expressão relacional ou lógica.
 - Um programa pede a idade de uma pessoa. Se a idade for maior ou igual a 18, ele imprime "Você é maior de idade". Caso contrário, imprime "Você é menor de idade".
- Um **bloco de comandos** é um conjunto de instruções agrupadas pela **indentação** dentro de um comando anterior seguido de **dois pontos**.
- A indentação é feita em geral com **4 espaços em branco** antes de cada comando que deve estar dentro do bloco.
- O principal comando condicional é o `if`, cuja sintaxe é:

```
1 if (expressão relacional ou lógica):  
2     comandos
```

- Os comandos são executados somente se a expressão relacional ou lógica for **verdadeira**.

Exemplo: Verificando se um número é par

Faça um programa que determina se um valor inteiro é par:

```
1 a = int(input())  
2 if (a % 2 == 0):  
3     print("O número digitado é par")
```

Comandos condicionais

Uma variação do comando if é o if/else, cuja sintaxe é:

```
1 if (expressão relacional ou lógica):  
2     comandos executados se a expressão é verdadeira  
3 else:  
4     comandos executados se a expressão é falsa
```

Exemplo: Determinando o menor de dois números:

```
1 a = int(input("Digite um número: "))  
2 b = int(input("Digite um número: "))  
3 if (a < b):  
4     print("O menor número é:", a)  
5 else:  
6     print("O menor número é:", b)
```

Comando if dentro de outro if

Note que o if é um comando, e como tal pode aparecer dentro do bloco de comandos de outro if.

Exemplo: Usando apenas operadores relacionais, vamos escrever um programa que lê um número e verifica se este é:

- Par e menor que 100
- Par e maior ou igual a 100
- Ímpar e menor que 100
- Ímpar e maior ou igual a 100

```
1 a = int(input("Digite um número:"))
2 if (a % 2 == 0):                               # Se número for par
3     if (a < 100):
4         print("O número é par e menor do que 100")
5     else:
6         print("O número é par e maior ou igual que 100")
7 else:                                           # Se número for ímpar
8     if (a < 100):
9         print("O número é ímpar e menor do que 100")
10    else:
11        print("O número é ímpar e maior ou igual que 100")
```

Refatorando com operadores lógicos

Refatorar um código significa reestruturar o código existente sem alterar o seu comportamento externo.

Ou seja, o programa continua fazendo a mesma coisa, mas o código fica mais claro, mais organizado e/ou mais fácil de manter.

```
1 a = int(input("Digite um número:"))
2 if (a % 2 == 0) and (a < 100):
3     print("O número é par e menor do que 100")
4 if (a % 2 == 0) and (a >= 100):
5     print("O número é par e maior ou igual que 100")
6 if (a % 2 != 0) and (a < 100):
7     print("O número é ímpar e menor do que 100")
8 if (a % 2 != 0) and (a >= 100):
9     print("O número é ímpar e maior ou igual que 100")
```

Comandos condicionais

Lembre-se que o que define a qual bloco de comandos um comando pertence é a sua **indentação**.

```
1 if (cond1):  
2     if (cond2):  
3         comando1  
4 else:  
5     comando2
```

Quando o comando2 é executado?

Resposta: quando a cond1 for falsa.

Comandos condicionais

```
1 if (cond1):  
2     if (cond2):  
3         comando1  
4     else:  
5         comando2
```

Quando o comando2 é executado?

Resposta: quando a cond1 for verdadeira e cond2 for falsa.

Comandos condicionais

```
1  if (cond1):  
2      if (cond2):  
3          comando1  
4      else:  
5          comando2  
6  else:  
7      if (cond3):  
8          comando3  
9      else:  
10         comando4
```

Quando o comando4 é executado?

Resposta: quando cond1 for falsa e cond3 também for falsa.

Exemplo de indentação

```
1 a = 5
2 if (a > 3):
3     if (a < 7):
4         print("a")
5     else:
6         if (a > -10):
7             print("b")
8         else:
9             print("c")
```

O que será impresso?

Saída: a

Explicação: $a > 3$ é verdadeiro, $a < 7$ também é verdadeiro, então imprime "a".

Exemplo de indentação

```
1 a = -12
2 if (a > 3):
3     if (a < 7):
4         print("a")
5     else:
6         if (a > -10):
7             print("b")
8         else:
9             print("c")
```

O que será impresso?

Saída: (nada será impresso)

Explicação: $a > 3$ é falso, portanto nenhum bloco interno é executado.

Exemplo de indentação

```
1 a = 9
2 if (a > 3):
3     if (a < 7):
4         print("a")
5     else:
6         if (a > -10):
7             print("b")
8         else:
9             print("c")
```

O que será impresso?

Saída: b

Explicação: $a > 3$ é verdadeiro, $a < 7$ é falso, e $a > -10$ é verdadeiro, então imprime "b".

Exercício

1. A solução abaixo está correta para classificação de um número como par/ímpar e menor/menor que 100, como no exemplo visto anteriormente? Caso seja, justifique. Caso não seja, indique os erros.

```
1 print("Digite um número:")
2 a = int(input())
3 if(a % 2 == 0) and (a < 100):
4     print("O número é par e menor do que 100")
5 else:
6     if(a >= 100):
7         print("O número é par e maior ou igual que 100")
8     if(a % 2 != 0) and (a < 100):
9         print("O número é ímpar e menor do que 100")
10    else:
11        if(a >= 100):
12            print("O número é ímpar e maior ou igual que 100")
```

2. Escreva um programa lê três números e imprime o maior deles.
3. Escreva um programa lê três números e os imprime em ordem (ordem crescente).

Obs.: Façam e me expliquem na próxima aula prática.

Comandos Condicionais if-else

Vamos fazer um programa que calcula a área de três tipos de objetos geométricos: **quadrado**, **retângulo** e **círculo**.

- Primeiramente, deve ser lido um caractere que indica o tipo de objeto:
 - ▶ 'q' para quadrado
 - ▶ 'r' para retângulo
 - ▶ 'c' para círculo
- Em seguida, devem ser lidas as dimensões do objeto:
 - ▶ Para um **quadrado**, deve ser lido o tamanho de um lado.
 - ▶ Para um **retângulo**, devem ser lidos os tamanhos dos dois lados.
 - ▶ Para um **círculo**, deve ser lido o raio.
- O programa deve calcular a área e imprimi-la.
- Se o usuário digitar um caractere diferente de 'q', 'r' ou 'c', o programa deverá imprimir uma **mensagem de erro**.

Comandos Condicionais if-else

Exemplo: Cálculo de área sem uso de if-else:

```
1 print("Digite uma opção (q, r, ou c):")
2 a = input()
3 if (a == "q"):
4     l = input("Digite o tamanho do lado do quadrado: ")
5     l = float(l)
6     print("A área é : %.2f" % (l * l))
7 if (a == "r"):
8     l1 = input("Digite o tamanho de um lado do retângulo: ")
9     l1 = float(l1)
10    l2 = input("Digite o tamanho do outro lado do retângulo: ")
11    l2 = float(l2)
12    print("A área é : %.2f" % (l1 * l2))
13 if (a == "c"):
14     r = input("Digite o tamanho do raio: ")
15     r = float(r)
16     print("A área é : %.2f" % (3.1415 * r * r))
17 if (a != "q" and a != "r" and a != "c"):
18     print("Opção inválida!")
```

Reescreva o programa acima utilizando a estrutura if-else.

Comandos Condicionais if-else

Refazendo o programa utilizando if-else:

```
1 print("Digite uma opção (q, r, ou c):")
2 a = input()
3 if (a == "q"):
4     l = input("Digite o tamanho do lado do quadrado: ")
5     l = float(l)
6     print("A área é : %.2f" % (l * l))
7 else:
8     if (a == "r"):
9         l1 = input("Digite o tamanho de um lado do retângulo: ")
10        l1 = float(l1)
11        l2 = input("Digite o tamanho do outro lado do retângulo: ")
12        l2 = float(l2)
13        print("A área é : %.2f" % (l1 * l2))
14    else:
15        if (a == "c"):
16            r = input("Digite o tamanho do raio: ")
17            r = float(r)
18            print("A área é : %.2f" % (3.1415 * r * r))
19        else:
20            print("Opção inválida!")
```

Estrutura de repetição com múltiplos ramos

- As estruturas de seleção **if-elif** e **if-elif-else** são utilizadas quando se deseja executar uma entre vários blocos, dependendo dos valores das expressões.

Cálculo da área de objetos, utilizando a construção if-else-if:

```
1 print("Digite uma opção (q, r, ou c):")
2 a = input()
3 if (a == "q"):
4     l = input("Digite o tamanho do lado do quadrado: ")
5     l = float(l)
6     print("A área é : %.2f" % (l * l))
7 elif (a == "r"):
8     l1 = input("Digite o tamanho de um lado do retângulo: ")
9     l1 = float(l1)
10    l2 = input("Digite o tamanho do outro lado do retângulo: ")
11    l2 = float(l2)
12    print("A área é : %.2f" % (l1 * l2))
13 elif (a == "c"):
14    r = input("Digite o tamanho do raio: ")
15    r = float(r)
16    print("A área é : %.2f" % (3.1415 * r * r))
17 else:
18    print("Opção inválida!")
```

Vários if seguidos ou if-elif-else?

O uso de vários if seguidos pode ser menos eficiente do que usar if-elif-else. Por quê?

- Quando usamos vários if, todas as condições são verificadas, mesmo que a primeira já seja verdadeira.
- Com if-elif-else, a verificação para assim que uma condição verdadeira é encontrada.

Conclusão: Para condições mutuamente exclusivas, prefira if-elif-else.

```
1 x = 2
2 if x == 1:
3     print("um")
4 if x == 2:
5     print("dois")
6 if x == 3:
7     print("três")
```

```
1 x = 2
2 if x == 1:
3     print("um")
4 elif x == 2:
5     print("dois")
6 elif x == 3:
7     print("três")
8 else:
9     print("outro número")
```

Mesmo que o programa descubra que $x==2$ é verdadeiro, ele ainda verifica as outras condições ($x==1$ e $x==3$).

Quando o programa encontra $x==2$, ele para imediatamente e não verifica as condições seguintes.

Comparando estruturas de decisão

Exemplo com vários if (verifica tudo):

```
1 if cond1:  
2     ...  
3 if cond2:  
4     ...  
5 if cond3:  
6     ...
```

Exemplo com if-elif-else (verifica até satisfazer):

```
1 if cond1:  
2     ...  
3 elif cond2:  
4     ...  
5 elif cond3:  
6     ...
```

Essa abordagem torna o programa mais claro e, em alguns casos, mais rápido.

Comando split()

O comando `split()` é usado para dividir uma string em partes, com base em um separador (por padrão, o **espaço em branco**).

A saída é uma **lista**.

Sintaxe:

```
1 string.split(separador)
```

Exemplo:

```
1 texto = "maçã banana uva"  
2 frutas = texto.split()  
3 print(frutas)  
4 # Saída: ['maçã', 'banana', 'uva']
```

Você também pode usar outro separador:

```
1 data = "10/08/2025"  
2 partes = data.split("/")  
3 print(partes)  
4 # Saída: ['10', '08', '2025']
```

Estrutura de Seleção com Múltiplos Ramos

Exemplo de uso do `split()` com comandos condicionais:

```
1 valores = input("Entre com dois inteiros positivos: ").split()
2 x = int(valores[0])           # primeira substring da lista
3 y = int(valores[1])           # segunda substring da lista
4 op = input("Informe o operador (+, -, *, / ou **): ")
5
6 if op == "+":
7     resultado = x + y
8 elif op == "-":
9     resultado = x - y
10 elif op == "*":
11     resultado = x * y
12 elif op == "/":
13     resultado = x / y
14 elif op == "**":
15     resultado = x ** y
16
17 print(x, op, y, "=", resultado)
```

`valores[i]` retorna o elemento na posição `i` da string alocada na variável `valores`, começando da posição 0.

Obs.: Veremos nas próximas aulas outros comandos e operações em strings.

Estrutura de Seleção com Múltiplos Ramos

```
1  valores = input("Entre com dois inteiros positivos: ").split()
2  x = int(valores[0])
3  y = int(valores[1])
4  op = input("Informe o operador (+, -, *, / ou **): ")
5
6  if op == "+":
7      resultado = x + y
8  elif op == "-":
9      resultado = x - y
10 elif op == "*":
11     resultado = x * y
12 elif op == "/":
13     resultado = x / y
14 elif op == "**":
15     resultado = x ** y
16 else:
17     resultado = None
18
19 if resultado == None:
20     print(op, ": Operador inexistente!!")
21 else:
22     print(x, op, y, "=", resultado)
```